

Predicting Obesity Without Weight and Height
Matthew Tankersley

Tankersley.13@Wright.edu

Abstract: The goal of this paper is to identify if obesity levels can be identified without the two components to calculate BMI, weight or height. I trained a simple and complex model using a collected list of features. I was able to achieve low results from my simple model and moderate results using my complex model. Thus, I believe it is possible to identify obesity levels based off other factors besides BMI.

Keywords: Obesity, LogisticRegression, MLPClassifier, GridSearchCV, BaggingClassifier

GitHub Link: https://github.com/MTankersley01/CS7830_Obesity_Final_Project

1) Introduction

The goal of this paper is to find out if a participant's obesity level can be identified by using a group of features excluding weight and height. Weight and height are used to calculate BMI which is the indicator to decide a participant's level of obesity.

The dataset I am using is from archive.ics.uci.edu ([Reference 1](#)). This dataset was collected from Latin American countries ([Reference 2](#)). It contains a combination of personal information, daily habits, eating habits, family information, physical conditions, physical activity information and biological traits.

This is a supervised learning dataset. The dataset contains 1 class and 16 features. The class is divided into 7 levels of obesity. The features are a mix of binary values, integer values, and categorical data. There are 2111 rows in this dataset. 23% of the data was collected online from participants and the remaining 77% of the data is synthetic and was generated using SMOTE ([Reference 14](#)). The dataset was generated to guarantee class balance, thus all classes are roughly equal. The dataset was also preprocessed to remove missing and atypical data thus the dataset had no missing values and was standardized.

This dataset was chosen because I felt that it provided an interesting opportunity to see if information gathered from participants about their lives and history could identify how obese they were. I am curious if weight and height are necessary to predict how obese an individual is or can other factors of someone's life provide that information.

Preventing overfitting was particularly important to me and a major focus because of the small region this data was collected in as well as the majority of the data being synthetically generated.

I propose testing a simple logistic regression model and if that fails, a more complex Multi-layer Perceptron Classifier (MLP Classifier) model to tackle the problem of predicting a participant's level of obesity.

Using the scikit learn Logistic Regression model ([Reference 3](#)) and MLP Classifier model ([Reference 4](#)) to predicted different levels of obesity. I evaluated the obesity dataset ([Reference 1](#)) and achieved moderate results using the MLP Classifier model. Below I have provided extensive ablation studies to understand both model's components and why they were chosen.

2) Methodology

Problem Formulation:

Both the scikit learn logistic regression and MLP Classifier use the below problem formulation.

- **Loss function:** Categorical Cross-Entropy ([Reference 5](#))

$$L(y, \hat{y}) = - \sum_{i=1}^c y_i \log(\hat{y}_i)$$

- **Regularization:** L2-regularization (Ridge) - Penalty ([Reference 4](#))

$$\frac{\alpha}{2n} \|W\|_2^2$$

- **Multiclass Activation Function:** Softmax ([Reference 4](#))

$$\text{softmax}(z)_i = \frac{\exp(z_i)}{\sum_{l=1}^k \exp(z_l)}$$

The goal of both the logistic regression and MLP Classifier models are to reduce $L(Y, \hat{Y})$ where Y is the ground truth and $\hat{Y}$ is the predicted class.

Model Architecture:

- **Logistic Regression** ([Reference 3](#))

The modifiable parameters for the Scikit learn Logistic Regression model are `L1_ratio`, `C`, `dual`, `tol`, `fit_intercept`, `intercept_scaling`, `class_weight`, `random_state`, `solver`, `max_iter`, `verbose`, `warm_start`, `n_jobs`.

- **MLP Classifier** ([Reference 4](#))

The modifiable parameters for the Scikit learn MLP Classifier model are `Hidden_Layer_Sizes`, `Activation`, `Solver`, `Alpha`, `Batch_size`, `Learning_rate`, `Learning_rate_init`, `Power_t`, `Max_iter`, `Shuffle`, `Random-state`, `Tol`, `Verbose`, `Warm_start`, `Momentum`, `Nesterovs_momentum`, `Early_stopping`, `Validation_fraction`, `Beta_1`, `Beta_2`, `Epsilon`, `N_iter_no_change`, `Max_fun`.

Each solver for the MLP Classifier has specific parameters that can be adjusted from the list of all parameters above:

- The **Adam** solver has `hidden_layer_sizes`, `activation`, `alpha`, `batch_size`, `learning_rate_init`, `max_iter`, `shuffle`, `random_state`, `tol`, `verbose`, `warm_start`, `early_stopping`, `validation_fraction`, `beta_1`, `beta_2`, `epsilon`, `n_iter_no_change`.
- The **SGD** solver has `hidden_layer_sizes`, `activation`, `alpha`, `batch_size`, `learning_rate`, `power_t`, `learning_rate_init`, `max_iter`, `shuffle`, `random_state`, `tol`, `verbose`, `warm_start`, `momentum`, `nesterovs_momentum`, `early_stopping`, `validation_fraction`, `n_iter_no_change`.
- The **LBFGS** solver has `hidden_layer_sizes`, `activation`, `alpha`, `batch_size`, `max_iter`, `random_state`, `tol`, `verbose`, `warm_start`, `validation_fraction`, `max_fun`.

- **Models:**

See figure 2 for visualizations depicting the models described above to a better understand their outputs.

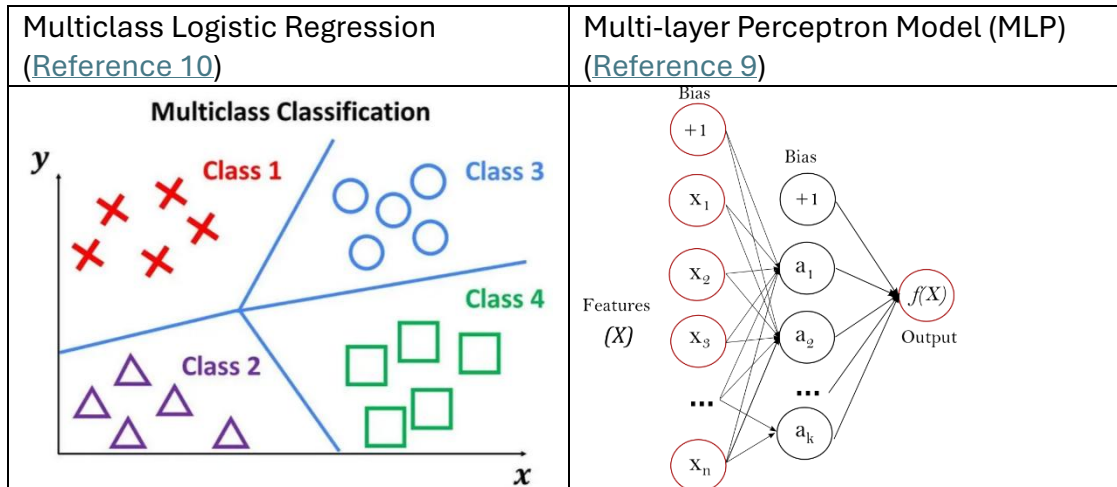


Figure 1

The following sections will discuss the MLPClassifier, this is important to determine which of the three solvers performed best.

Training Procedure:

I swept the Scikit Learn Logistic Regression and MLP Classifier documentation for all relevant adjustable parameters and used GridSearchCV to test those parameters ([Reference 7](#)). GridSearchCV implements K-fold cross validation using 5 folds for each combination of parameters. GridSearchCV runs a test using each combination of provided parameters and outputs the best set of parameters based on the default metric for the model. Both Logistic Regression and MLP Classifier use accuracy and thus accuracy was used to distinguish the best set of parameters. Accuracy is an acceptable metric to use because of the even class distribution. The results of the GridSearchCV were considered to be indisputable.

The Logistic Regression model was simple enough that I could test every parameter I wanted quickly. All applicable solvers, different epochs levels, different tolerance levels, and regularization values were tested. The MLP Classifier model was complicated and was divided into 3 separate models. One for each solver testing as many of the parameters that my system could handle. These included Hidden layer length and neuron sizes, activation functions, alpha levels, batch sizes, learning rates, epochs, convergence stopping criteria, and tolerances.

BaggingClassifier ([Reference 8](#)) was used to run the MLP Classifier models N times with bootstrapping and average the results in order to obtain a consistent output. I felt that this was necessary because I noticed different metrics using the same models depending on my train test split outcome. I believe this is because my data is small and contains noise.

I have a low end computer and training large amounts of parameters caused my model to fail to train in an acceptable time. As you can see from figure 3 below, this model has not finished after 1310 minutes. As my models were not able to finish training in a reasonable amount of time, I had to drastically reduce

the parameters I tested. My reduced parameters included higher learning rates, higher alpha values, lower batch sizes, and reducing the number of epochs.

```
#Set parameters to test
SGDparameters = {
    'hidden_layer_sizes': [(100,), (18,), (512,), (32, 16), (18, 18), (50, 25), (128, 64), (128, 128), (16, 12, 8), (256, 128, 64), (32, 16, 8, 4)],
    'activation': ['relu', 'logistic', 'tanh', 'identity'],
    'learning_rate': ['constant', 'invscaling', 'adaptive'],
    'solver': ['sgd'],
    'alpha': [0.0001, 0.01, 0.1, 10],
    'batch_size': ['auto', 32, 128, 512],
    'learning_rate_init': [0.001, 0.1, 10],
    'n_iter_no_change': [10, 25, 50]
}

#Set hard coded parameters for the mode, these parameters are not ment to change. Early stop resolves overfitting issues.
SGDgridSearchMLPNN = MLPClassifier(max_iter=10000, early_stopping=True)

#Default cross validation is 5
#The default scoring metric from MLPClassifier is used, this metric is Accuracy which should be acceptable as the classes are all similar sizes
SGDgridModel = GridSearchCV(SGDgridSearchMLPNN, SGDparameters)

#Parameter testing starts here
SGDgridModel.fit(xMLPTrainingData, yMLPTrainingData)

print("Best Parameters:", SGDgridModel.best_params_)
print("Best Accuracy Score:" SGDgridModel.best_score_)

1310m 48.8s
```

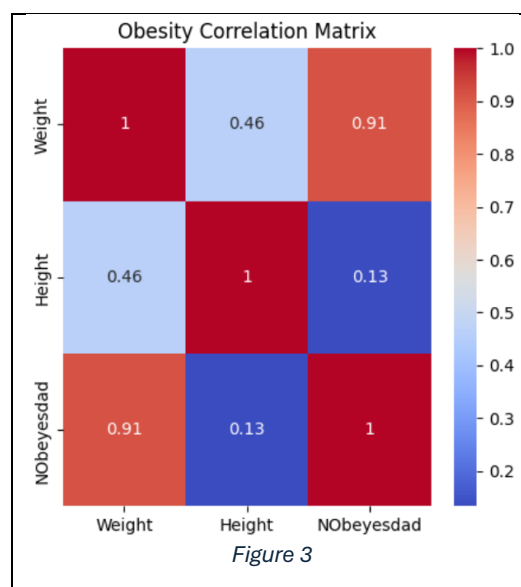
Figure 2

3) Experimental Setup

Dataset

The dataset I used was downloaded from archive.ics.uci.edu ([Reference 1](#)), sciencedirect.com ([Reference 11](#)) provided further details about the dataset.

Of the 16 features, 2 of them (weight and height) are used to compute the target class (BMI). As these 2 features are used to compute the class, they have been removed from the list of features used to train and test my models. Please see figure 1 below and view just how correlated Weight is with the target class NObeyesdad.



The binary features were encoded to have values 0 and 1. The categorical features were encoded to have values between 0 ~ N where 0 was the lowest frequency and N was the highest frequency. The transportation feature (MTRANS) has undergone One Hot Encoding ([Reference 17](#)), thus breaking it out into 5 individual features. After this was done, I was noticed that MTRANS_Automobile was highly negatively correlated (-.91) with MTRANS_Public_Transportatoin and thus removed from the features. My criteria for removing a feature were positive and negative correlation greater then or equal to .9.

The target class is a multiclass divided into 7 separate increasing levels of obesity: Insufficient_Weight, Normal_Weight, Overweight_Level_I, Overweight_Level_II, Obesity_Type_I, Obesity_Type_II

Permutation importance ([Reference 13](#)) was used to identify what features were most important for this experiment. In order, the most important features were Gender, Age, do you eat any food between meals (CAEC), do you usually eat vegetables in your meals (FCVC), and how often do you drink alcohol (CALC).

The remaining features were then normalized using the standard scaler ([Reference 6](#)).

I used a standard train / test split of 80% / 20% for my data. The BaggingClassifier used for the MLP Models further used bootstrapping on the training data roughly splitting the training data into a 63/37 validation split ([Reference 15](#)). When early stop was set on the MLP Classifier an additional 10% validation split was used for training ([Reference 4](#)).

	Train (80%)	Test (20%)	Class
Obesity Dataset	1688	423	7

Evaluation Metrics:

Accuracy was primarily used to measure model performance because of the equal class distribution. Macro average F1 was used as a second measure of model performance to double check the accuracy score. Macro average precision and recall were also taken into consideration. Individual F1, precision, and recall scores were reviewed to see where the model performed poorly as well as the AUC score and ROC graph to see if the model could perform better at different thresholds.

Implementation Details

My testing pipeline involved collecting a comprehensive list of parameters from the scikit learn documentation page and feeding the relevant parameters through GridSearchCV. GridSearchCV used the training accuracy to report the best model parameters. Those parameters were then used on the testing data through the BagerClassifier to receive an averaged accuracy, precision, recall, and F1 scores. I then used the same method on the training data to receive an average accuracy which I compared against the testing accuracy. If there was greater than a 10% difference in accuracy, I considered the model to have overfitted. Overfitting models were dropped from consideration for best model.

Final Implementation:

- **Solver:** adam
- **Alpha:** 0.01
- **Hidden layer size:** (16,)

- **Activation:** logistic
- **Max iter:** 500
- **Shuffle:** True
- **Random state:** None
- **Tol:** 0.0001
- **Learning rate init:** 0.1
- **N iter no change:** 10
- **Batch size:** 32
- **Early stopping:** True
- **Validation fraction:** .1
- **Beta 1:** 0.9
- **Beta 2:** 0.999
- **Epsilon:** 0.00000001
- **Warm start** = False
- **CPU:** AMD Ryzen 5 4600G

Implementation Justification:

- **Features:** Weight and height were removed because a requirement for this paper is that they be removed. MTRANS_Automobile was also removed because it had a .91 correlation with MTRANS_Public_Transportation.
- **Encoding:** Encoding was done in a way that preserved ordinal data.
- **Scalar:** Standard Scalar was used because the MinMaxScalar produced reduced metrics across all my models. See bottom of code for example using MinMaxScaler on my best MLP Classifier model.
- **GridSearchCV:** Used to find a model with the best parameters. This was done because of limitations to the equipment I had available. GridSearchCV was ran twice across 3 different optimizers. Once with many parameters where I noticed significant overfitting, and a second time with reduced parameters specifically tailored to resolve the overfitting issue.
- **Hidden Layers:** Changed from (100,) to (16,). Small hidden layers were used. I tested many hidden layers with more neurons and more layers but found large problems with overfitting.
- **Activation Function:** Set to logistic, GridSearchCV identified this activation function as the best function.
- **Solver:** Adam was selected. This solver produced the best metrics without having the overfitting problems experienced with the other solves.
- **Alpha:** Increased to 0.01 from 0.001, lower value was contributing to overfitting.
- **Batch_size:** Reduced to 32 from 200, higher value was contributing to overfitting.
- **Learning_Rate_init:** Increased to 0.1 from 0.001, this caused increased metrics.
- **Max_iter:** Increased to 500 to allow the model more time to converge. I received many errors when I kept this at the default value of 200. 500 was chosen because at 800 and above iterations I started to notice overfitting.
- **Tol:** Kept at default value, changing this value didn't cause noticeable metric changes.
- **Early_stopping:** Set to True to help with the models overfitting.

- **N_iter_no_change:** Kept at the default value, increasing this value contributed to my models overfitting.
- **Validation_fraction:** Kept at default value, changing this value didn't have an impact on my model metrics.
- **Beta_1:** Kept at default value, reducing this value had a negative impact on model metrics.
- **Beta_2:** Kept at default value, reducing this value had a negative impact on model metrics.
- **Epsilon:** Kept at default value, changing this value didn't have an impact on my model metrics.
- **Untested / Left Default:**
- **Shuffle:** Kept at default value (True), I wanted randomization each time the model ran.
- **Random_state:** Kept at default value (False), I wanted randomization each time the model ran.
- **Verbose:** Kept at default value (False), this caused excessive noise and I already have a graph for the loss data.
- **validation_fraction:** Kept at default value (False), I didn't want the previous call being used reused for fitting the model.

4) Results and Discussion

Main Results:

The Macro Average metrics will be used to discuss the models overall and used in my dataset summaries.

The individual class precision, recall, and F1 metrics will be used to discuss areas where the models excelled and failed.

Overfitting was decided by comparing training and testing metrics, primarily accuracy being 10% or further apart. Accuracy was used because of my balanced classes, and the score was often comparable to the macro average f1 score as a second opinion.

The results from GridSearchCV gave me my parameters. GridSearchCV was indisputable.

Summary of Dataset				
Method	Accuracy	Macro Precision	Macro Recall	Macro F1
Base Logistic Regression	0.58	0.58	0.58	0.56
Base MLP Adam Solver	0.66	0.65	0.66	0.65
Base MLP SGD Solver	0.45	0.44	0.45	0.42
Base MLP LBFGS Solver	0.78	0.78	0.78	0.78
Best Model, MLP Adam Solver	0.74	0.73	0.74	0.73

The MLP Classifier model using the adam solver, parameters to reduce overfitting found through GridSearchCV, and fitted through the averaged results from the BaggingClassifier provided my best metrics. All other models, even after performing parameter reductions to reduce overfitting ([Reference 18](#)), still had problems overfitting to the training data. The adam solver was chosen because it was the

only model that didn't overfit. In the summary of dataset image above the base LBFGS model shows better metrics but overfitted to the training data with 100% in all metrics.

Ablation Study & Discussion

The logistic regression model was chosen because of how simple it was. If a simple model was capable of distinguishing obesity levels using this dataset then I wouldn't have to deploy a more complex model.

The logistic regression model was basic enough that I was able to test most parameters at the same time with GridSearchCV. Below I will discuss my baseline and the best model I was able to achieve.

Logistic Regression Results:

Logistic Regression Confusion Matrix				
	Accuracy	Macro Precision	Macro Recall	Macro F1
Baseline Logistic Regression	0.58	0.58	0.58	0.56
Best Logistic Regression	0.59	0.58	0.58	0.56

Logistic Regression Baseline Metrics:

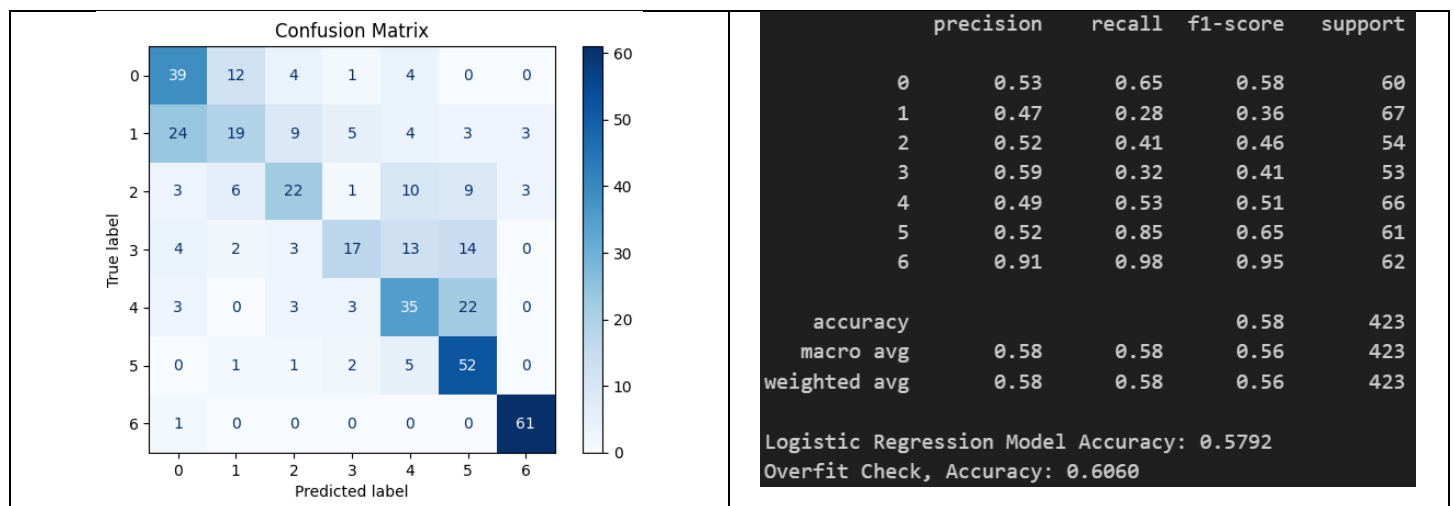


Figure 4

Logistic Regression Best Model Metrics:

This model used the parameters discovered through the GridSearchCV except for the solver and regularization type. I had to perform further testing with the saga solver because the saga solver is the only solver that supports Lasso regularization therefore Lasso regularization couldn't be tested in the GridSearchCV. Adding Lasso regularization to the GridSearchCV crashed the model because of the compatibility issues with other solvers.

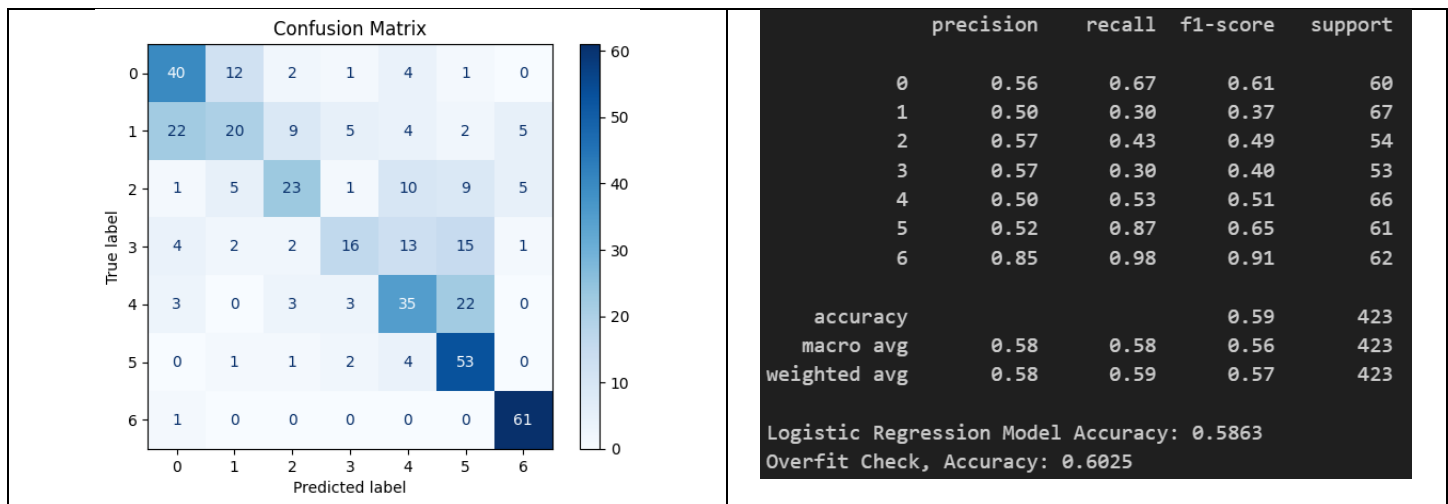


Figure 5

Logistic Regression Discussion:

Changes from the default baseline model to the best model: lbfgs solver changed to saga solver, Ridge regularization change to Lasso regularization, and tolerance changed from 0.0001 to 0.01. I do not believe the changes suggested by the GridSearchCV provided a relevant increase in metrics. The model appears to fail in similar ways between the base and best model outputs only increasing by a single point in accuracy.

You can tell that the final model struggled significantly to properly predict classes 1,2,3 and 4. The low recall for these individual classes shows that the model failed to predict these classes correctly, thus there are a lot of false negatives. The low precision for the individual classes 0 through 5 shows that the model was too eager to assign incorrect positive values, thus there were a lot of false positives. Overall, this model was only able to predict the highest obesity class consistently. Reviewing the confusion matrix you can see that the neighbor classes are consistently confused for each other. There even appears to be some overlap between classes that are not neighbors as shown by multiple 1 and 2 classes being predicted as 6.

MLP Classifier Results:

The MLP Classifier model was broken into 3 groups, one for each solver. The solver dictated the adjustable parameters. The three solvers were Adam, SGD, and LBFGS.

You can see from reviewing my code that I had to run GridSearchCV twice for each solver. This is because the first group of models had parameters that caused significant overfitting (accuracy significantly greater than 10% between training and testing data).

BaggingClassifier was used to fit the model. This function performs a bootstrapped fit of the model N times then averages them together. I used 100 for the estimator value, essentially running the model 100 times and taking the average to calculate my predictions from. BaggerClassifier didn't allow for printing my loss curve and was implemented after my baseline models were already built. Loss curve graphics have been provided for my baseline models but not my best models.

All base MPL Classifier metrics were taken using early stop with a 10% validation split, this was the only changed parameter from the model defaults. Without early stop the models turned out massively overfitted. Lbfgs does not support early stop and thus was not used.

MLP Classifier Adam Solver Results:

Adam Solver MLP Classifier Metrics				
	Macro Accuracy	Macro Precision	Macro Recall	Macro F1
Baseline Adam MLP Classifier	0.66	0.65	0.66	0.65
Best Adam MLP Classifier	0.74	0.73	0.74	0.73

Baseline MLP Classifier Adam solver model metrics:

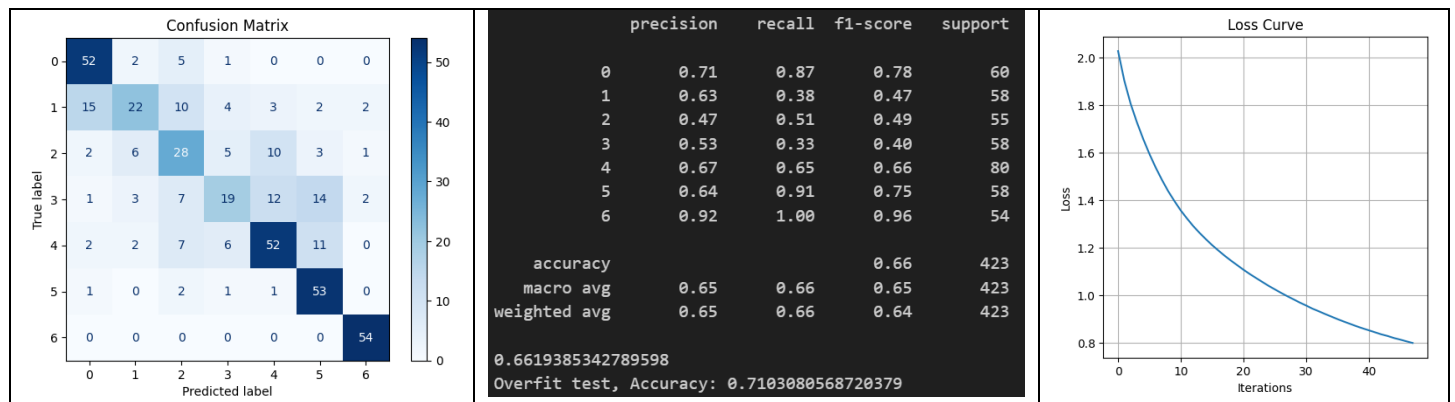


Figure 6

Best MLP Classifier Adam solver model metrics:

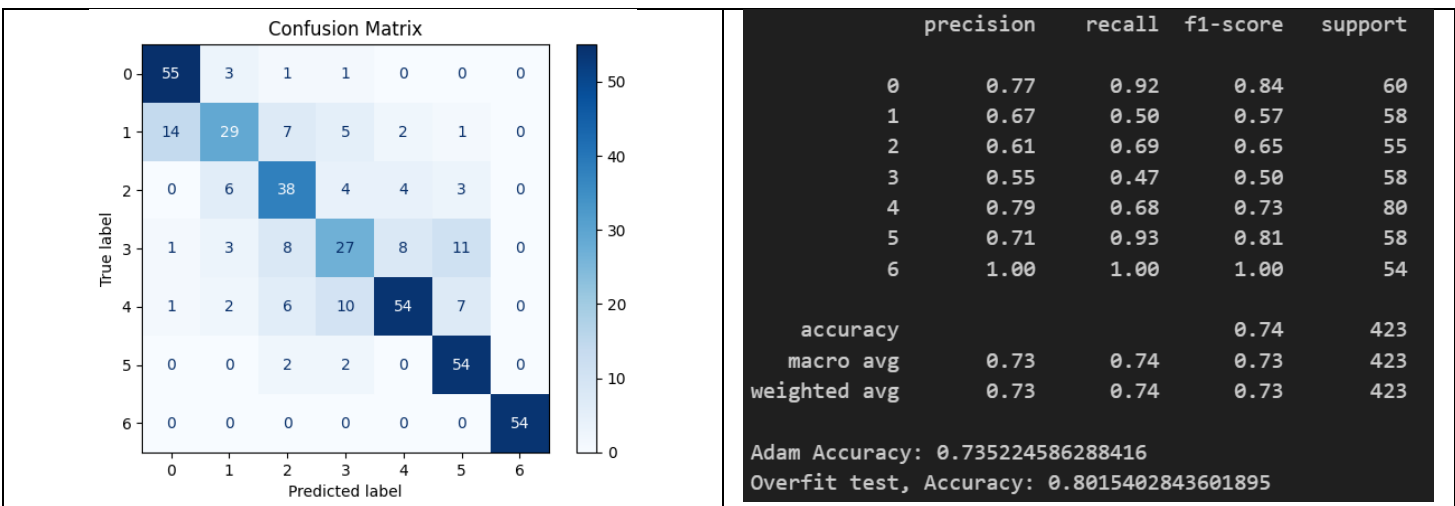


Figure 7

MLP Classifier Adam Solver Model Discussion:

Changes from the baseline model: Reduced hidden_layer_size from (100,) to (16,), increased alpha from 0.0001 to 0.01, changed the activation function from relu to logistic, increase max iterations from 200 to 500, and reduced the batch size from 200 to 32.

You can see clear improvement between the baseline model and the best model in every metric. Every single individual class precision, recall, and f1 scores as well as accuracy increased. This model was also the only model to stay within my overfitting criteria.

The best model metrics shows clear limitations computing classes 1, 3 and to a lesser extent 2. The Confusion matrix shows that the model struggled with neighboring classes.

Overfitting reduced from base mode at 13% and best model at 7%. This reduction can be contributed to the changes in hidden_layer_size, alpha, and batch sizes.

The loss curve shows that the model failed to converge. Allowing the model to converge almost always resulted in severe overfitting at high epochs. With early stop enabled the loss had not improved more than 0.0001 in 10 epoch causing the model to stop fitting.

MLP Classifier SGD Solver Results:

SGD Solver MLP Classifier Metrics				
	Accuracy	Precision	Recall	F1
Baseline SGD MLP Classifier	.45	.44	.45	.42
Best SGD MLP Classifier	.79	.79	.80	.79

Baseline MLP Classifier SGD solver model metrics:

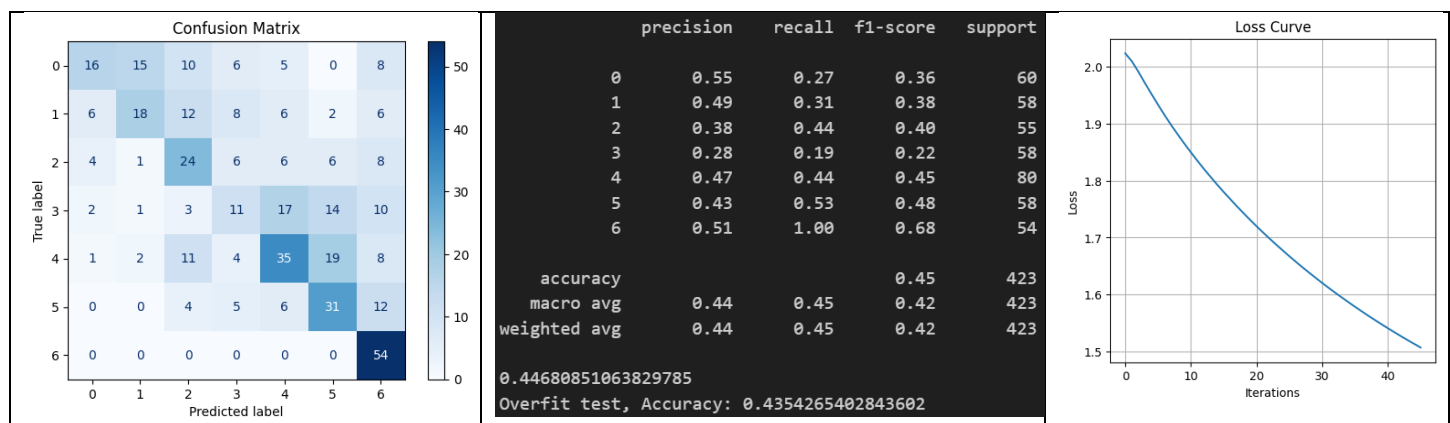
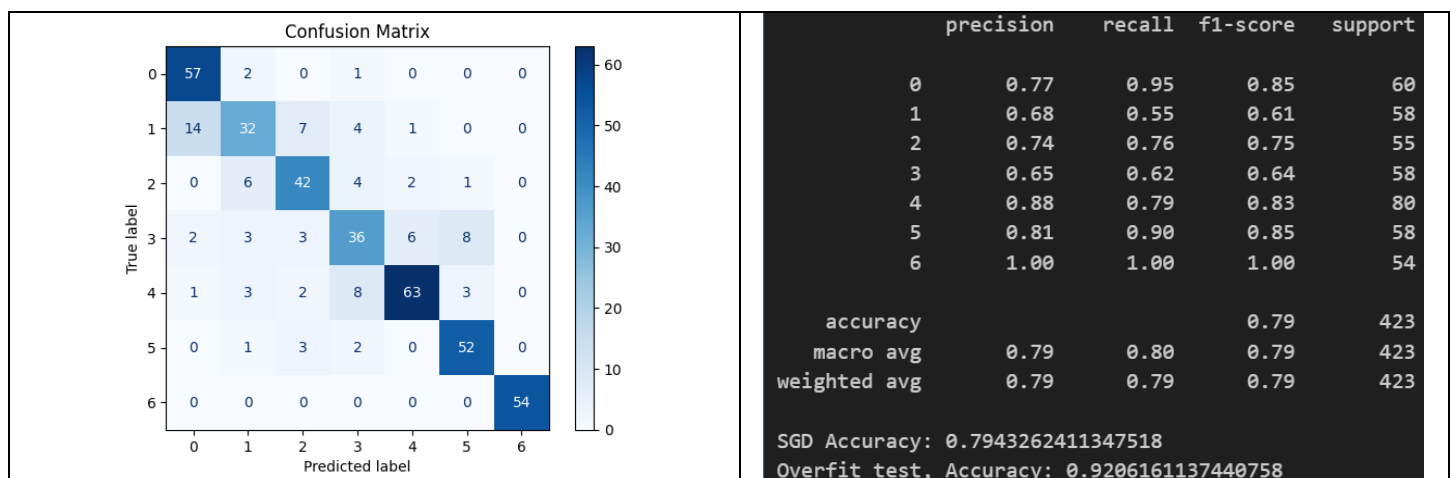


Figure 8

Best MLP Classifier SGD solver model metrics:



MLP Classifier SGD Solver Model Discussion:

Changes from the baseline model: Reduced the size of the first hidden layer node size from 100 to 16 and added a second layer 8, increased alpha from 0.0001 to 0.1, changed the activation layer from relu to tanh, increased the max iterations from 200 to 500, changed the learning rate from constant to adaptive, and decreased the batch size from 200 to 32.

You can see clear improvement between the baseline model and the best model in every metric. This model also has the highest increase in precision, recall, F-1 score, and accuracy among all models. I believe that increasing the number of layers of the hidden network and increasing the epochs contributed the most.

The best model metrics show clear limitations computing classes 1 and 3. The Confusion matrix shows that the model struggled with neighboring classes.

Overfitting increased from base model at 1% to best model at 13%. Unfortunately, I could not solve the overfitting issue. Alpha was increased and the batch size was decreased in order to resolve overfitting.

The loss curve for the baseline model shows that it failed to converge. I believe the base parameters of the MLP Classifier model itself were not robust enough to handle the SGD solver. With early stop enabled the loss did not improve more than 0.0001 in 10 epoch causing the model to stop fitting where it did.

MLP Classifier LBFGS Solver Results:

LBFGS Solver MLP Classifier Metrics				
	Accuracy	Macro Precision	Macro Recall	Macro F1
Baseline LBFGS MLP Classifier	0.78	0.78	0.78	0.78
Best LBFGS MLP Classifier	0.80	0.79	0.80	0.79

Baseline MLP Classifier LBFGS solver model metrics:

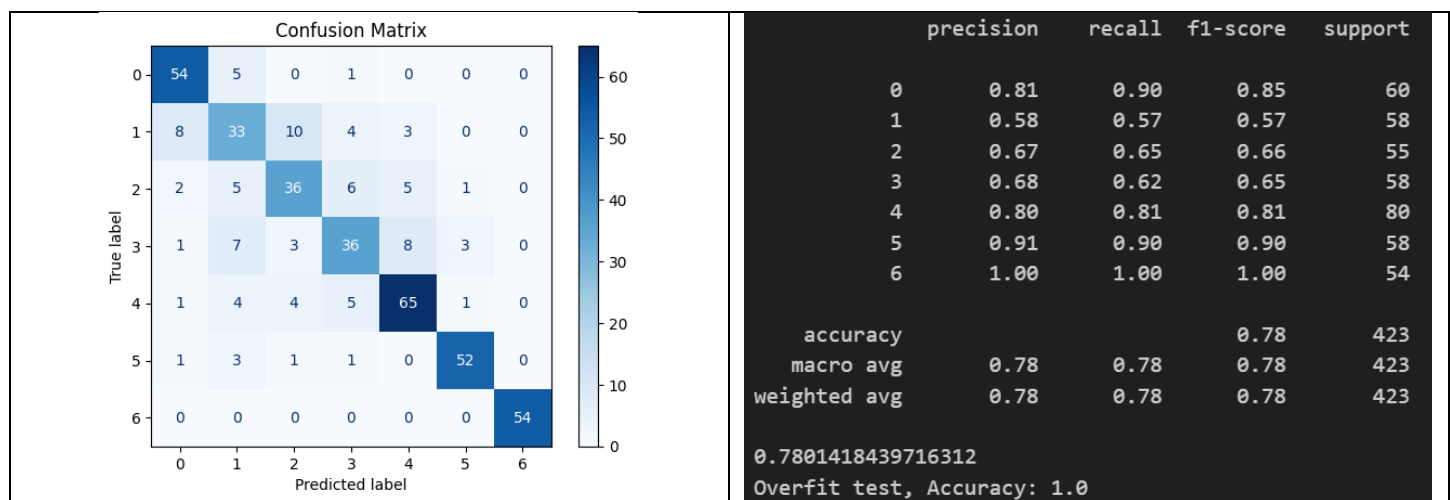


Figure 9

Best MLP Classifier LBFGS solver model metrics:

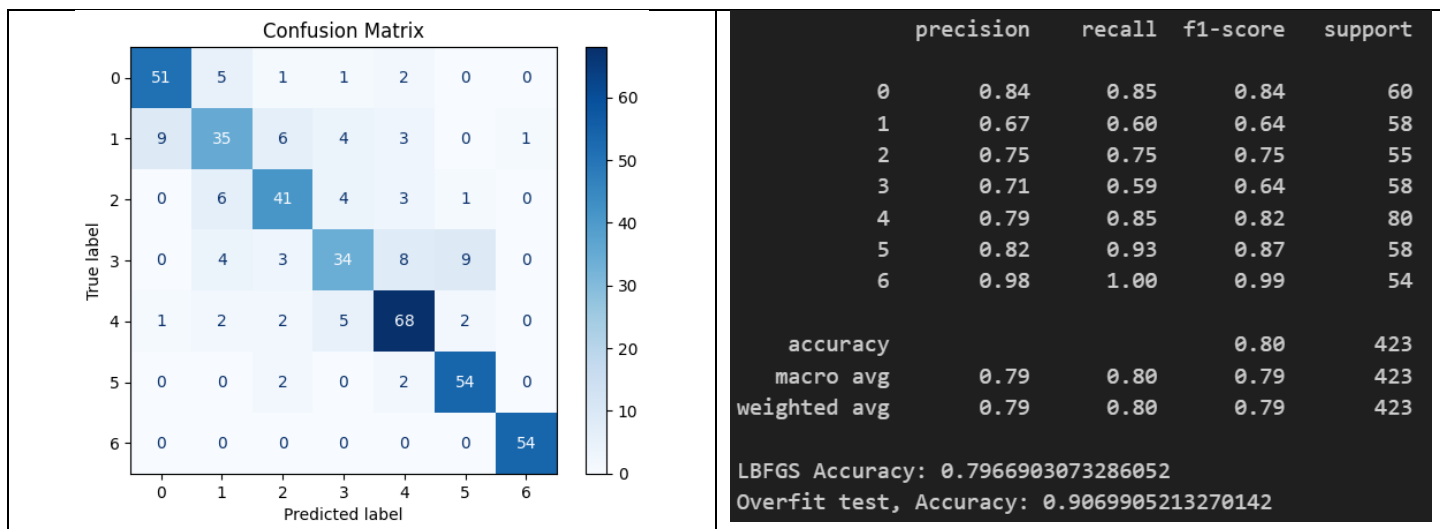


Figure 10

MLP Classifier LBFGS Solver Model Discussion:

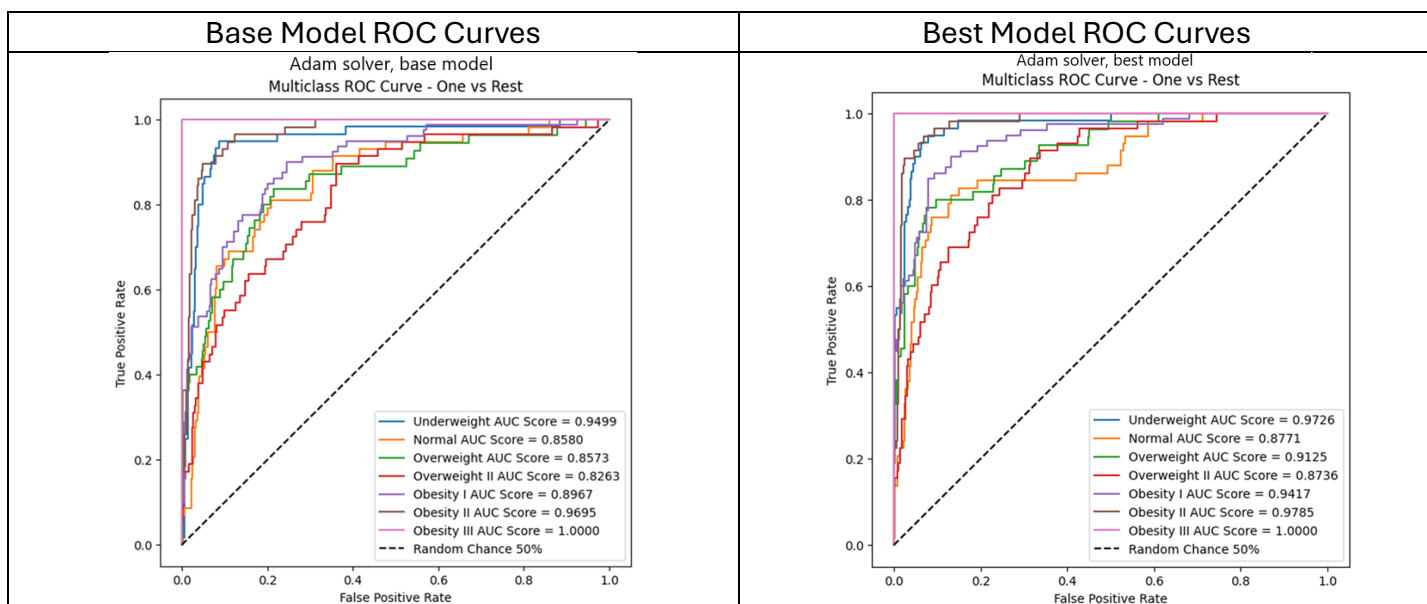
Changes from the baseline model: Reduced the size of the first hidden layer node size from 100 to 16 and added a second layer 8, increased alpha from 0.0001 to 0.5, changed the activation layer from relu to logistic, and increased the max iterations from 200 to 500.

The metrics for the base model and best model are only slightly better with accuracy increasing by 2, precision increasing by 1, recall increasing by 2, and f1 score increasing by 1.

The best model metrics show clear limitations computing classes 1 and 3. The Confusion matrix shows that the model struggled with neighboring classes, especially class 1 which had a prediction as high as 6.

Overfitting decreased from the base model at 22% to the best model at 11%. You can see a clear improvement in overfitting, but I was not able to solve the issue. This reduction can be contributed to the changes in hidden_layer_size, alpha, and batch sizes.

ROC Curve & AUC Results:



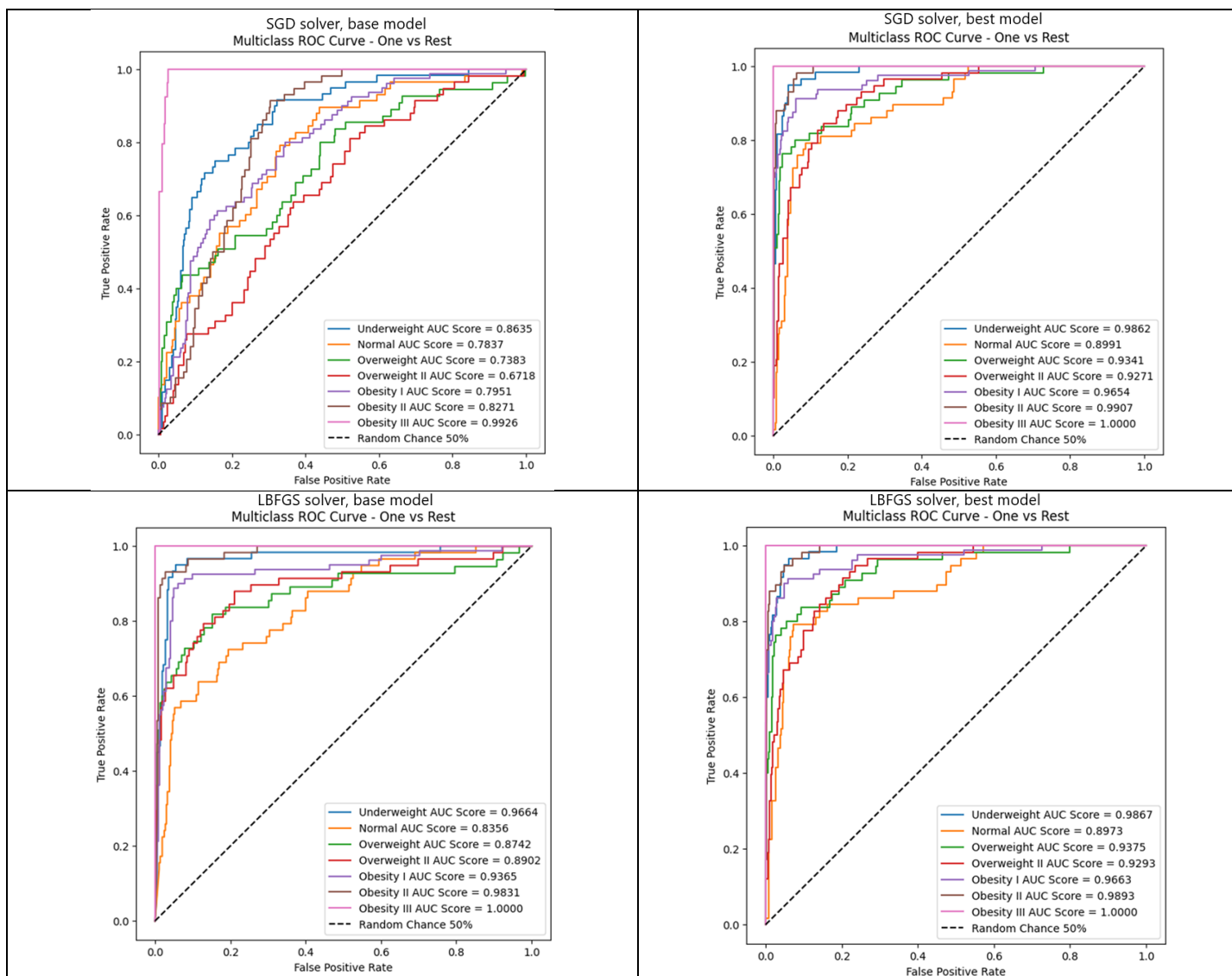


Figure 11

ROC Curve & AUC Discussion:

I produced my ROC curves using One vs Rest because of my multi class data. For all models, including the models that were poor like the base SGD model, the AUC values were high. The curves themselves even turned out well with none below the 50% line. I believe that using the One vs Rest ROC curve was a mistake because of the way the One vs Rest foundationally works. Since there were so many classes there was a high number of true negatives causing my curves to be skewed ([Reference 12](#)).

Overall Best Model Results:

Adam Solver MLP Classifier Metrics				
	Macro Accuracy	Macro Precision	Macro Recall	Macro F1
Best Adam MLP Classifier	0.74	0.73	0.74	0.73

Overall Best Model Discussion:

The MLPClassifier model using the adam solver was chosen as my best model. This is because it had the highest metrics of any model that didn't overfit per my 10% accuracy rule.

There was a clear overfitting issue shown throughout nearly all my models, even after attempting many times to solve the issue through parameter modification. I believe to solve this issue the amount of data and the diversity of the data needs to be increased. I believe that 77% of the data being synthetically produced using SMOTE also contributed to overfitting ([Reference 16](#)).

This model has a few obvious limitations. I believe that 7 classes is too many for this model to be able to reliably identify, specifically the middle classes. Every model underperformed to predict class 1 and 3. Nearly every model predicted the extremes well, 0, 6 and 7. If you were underweight or the highest level of Obese then the model performed very well. If you were somewhere in the middle, then the model struggled. I believe this is because the features were overlapping for middle value classes and this can be seen by reviewing the confusion matrices.

This limits the model's usefulness where it can only reliably predict underweight or normal participants and the higher end of obese participants. This model would do a poor job at helping someone predict if they were becoming obese as their patterns would overlap with the surrounding features.

5) Technical Challenges

Given more time I would have loved to do more testing with different configurations for the target data. There are 2 specific ways I would have liked to test the data. Instead of having 7 classes I would have like to consolidate them into 3 classes, normal, overweight, and obese. Second, I would change the data to a binary class where there is only normal and overweight. I believe that the models would have worked better had there been less classes.

The biggest challenge with this dataset was overfitting. I based my overfitting on the training accuracy vs the testing accuracy. If the accuracy were 10% or more apart, I considered the model overfitted to the training data. This paper taught me how hard it is to balance improving your metrics and preventing the model from fitting to the training data. I believe the small size of the dataset, amount of synthetic data, and lack of diversity contributed to overfitting.

I found my hardware to be limiting. My computer was purchased exclusively for school, it is not a high end computer. At first, I was testing every possible parameter I could think of but found that my computer could not support such large models. I had one GridSearchCV take almost 2 days to finish and that was just for a single solver! This caused me to have to drastically reduce my parameters. If I had a higher power computer I would have enjoyed testing more parameter options.

6) Conclusion

I was successfully able to build a multi-layer perception classifier model to achieve 0.74 accuracy, 0.73 macro average precision, 0.74 macro average recall, and 0.73 macro average f1-score when predicting participants level of obesity. This was achieved using a dataset where the features used to compute the target class were excluded. GridSearchCV provided the appropriate parameters and BaggingClassifier guaranteed consistency. Together these methods were used to achieve these scores.

Of the 7 distinct classes my model was able to successfully predict extreme cases well, 0, 5 and 6. My model underperformed at predicting cases in the middle 1, 2, 3 and 4.

I believe that these metrics show moderate promise for future analysis. I believe that this is a good foundation for improvements that could be obtained through increasing the amount of data and transforming the class distribution.

I believe that it is possible to build a model that predicts levels of obesity without the components of calculating obesity, weight and height. I believe that the MLP Classifier model is a good starting point and useful model, but I believe further testing could achieve even greater metrics than I was able to.

References:

- 1) <https://archive.ics.uci.edu/dataset/544/estimation+of+obesity+levels+based+on+eating+habits+and+physical+condition>
- 2) <https://www.britannica.com/place/Latin-America>
- 3) https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- 4) https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
- 5) <https://www.geeksforgeeks.org/deep-learning/categorical-cross-entropy-in-multi-class-classification/>
- 6) <https://www.geeksforgeeks.org/machine-learning/standardscaler-minmaxscaler-and-robustscaler-techniques-ml/>
- 7) https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
- 8) <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingClassifier.html>
- 9) https://scikit-learn.org/stable/modules/neural_networks_supervised.html
- 10) <https://medium.com/@jshaik2452/multi-class-logistic-regression-a-friendly-guide-to-classifying-the-many-4a590c2e6c26>
- 11) <https://www.sciencedirect.com/science/article/pii/S2352340919306985>
- 12) <https://towardsdatascience.com/precision-recall-curve-is-more-informative-than-roc-in-imbalanced-data-4c95250242f6/>
- 13) https://scikit-learn.org/stable/modules/generated/sklearn.inspection.permutation_importance.html
- 14) <https://www.jair.org/index.php/jair/article/view/10302>
- 15) <https://stats.stackexchange.com/questions/470646/bagging-size-of-the-aggregate-bags>
- 16) <https://pmc.ncbi.nlm.nih.gov/articles/PMC10789107/>
- 17) <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OneHotEncoder.html>
- 18) <https://eureka.patsnap.com/report-how-to-reduce-overfitting-in-multilayer-perceptrons>